
Person A — Data Pipeline Checklist

Key Questions to Address Before Building the Database

ML Factor Investing Project • S&P 500 • 2005–2024

Why this document exists. Before Person A writes a single line of data-pulling code, they must have a clear answer to every question below. These are the issues that silently ruin equity backtests — survivorship bias, look-ahead bias, improper handling of missing data, and wrong universe construction. Any one of them invalidates every Sharpe ratio the team ever produces. Get these right on Day 1; everything else follows.

1. Universe: Which Stocks Do We Track?

Q1. Which stocks belong in our data pull?

Answer: *Every stock that was ever a member of the S&P 500 at any point between 2005 and 2024*, not the current 500. A one-time index membership is enough to justify pulling the stock's data. This produces a universe of approximately **1000 unique tickers** across the full period.

Q2. Why not just use today's S&P 500?

Survivorship bias. Using today's S&P 500 means your 2008 universe silently excludes Lehman Brothers, Bear Stearns, Washington Mutual, and every other company that went bankrupt. It also includes Tesla, Meta, and other giants that were not yet public or not yet large enough. You're handing your model the winners and hiding the losers. Result: Sharpe ratio inflated by 0.3–0.8 for free, and the strategy would collapse in production.

Q3. Where do I get the historical constituents list?

- **Best free option:** a maintained GitHub CSV such as `fja05680/sp500` or similar (search "*sp500 historical constituents csv*"). Provides monthly index membership snapshots going back decades.
- **Gold standard:** CRSP via WRDS if your university has access.
- **Verification:** cross-check against Wikipedia's "*List of S&P 500 companies*" (Selected changes table).

2. Point-in-Time Universe: Who Is Investable *Today*?

Q4. At a given rebalance date, which stocks can the strategy actually trade?

Answer: Only the stocks that were *actual members of the S&P 500 on that specific date*. Being in the index once is enough to be in the database; being in the index *on this date* is required to be in the investable universe.

Rule. Pull broadly, filter strictly.
Pull every ticker that ever touched the index (the database).
Filter at each rebalance to only tickers in the index on that date (the universe).

Q5. How is this implemented?

Maintain a separate `constituents.parquet` table with one row per (month-end, ticker) pair and a boolean `in_index` column. The backtest loads this table and uses it as a mask at every rebalance:

```
universe_t = constituents[constituents.date == rebalance_date][
    constituents.in_index == True
].ticker
```

Q6. What about index changes mid-month?

For a student project, use a **monthly snapshot** (month-end membership). If a stock joined mid-March, it becomes investable at the April 1 rebalance, not the day it joined. Introduces at most a one-month lag. Academic papers (Gu, Kelly & Xiu) effectively do this. Document it in the methodology section.

3. Missing Data Philosophy: NaN Is Information

Q7. What should a stock's price be on dates when it didn't yet exist?

Answer: NaN. Do not backfill, do not impute, do not interpolate. Let the NaN propagate through return and factor computation; the stock will naturally drop out of any universe that requires those values.

Q8. What should happen when a stock gets delisted (bankruptcy, acquisition)?

Answer: The stock has real prices up to its last trading day, then NaN afterwards. The backtest must realize the final return:

```
sell_price = prices[ticker].loc[:rebalance_date].dropna().iloc[-1]
period_return = sell_price / buy_price - 1
```

A Lehman position held through September 2008 realizes a -99% return. A Twitter position held through the October 2022 privatization realizes a $+22\%$ gain. The strategy *cannot* magically step out before delisting — that would be look-ahead.

Q9. What about missing fundamentals (much more common than missing prices)?

- **Forward-fill over time within each stock.** If book value is reported on Nov 5, 2020, it stays \$10B every day until the next quarterly report. This is the one place forward-filling is correct.
- **Do not cross-sectionally fill.** Never replace a missing value with the sector or industry average. A genuinely missing field stays NaN.
- **Apply a 45-day reporting lag.** A Q1 report (quarter-end March 31) is not publicly available until mid-May. Your code must enforce `fundamentals.shift(45_days)` before any use.

Look-ahead bias trap. Using fundamentals on their quarter-end date instead of their report date is the single most common look-ahead bug in student projects. It quietly inflates Sharpe by 0.2–0.5. Enforce the 45-day lag *inside* `data_loader.py` so no downstream code can accidentally violate it.

4. Raw Data Bundle: What Exactly to Pull

Q10. What raw fields must be in the database?

Person A pulls raw data, not features. The standard bundle below covers every factor in Gu, Kelly & Xiu (2020) and every factor Person B will plausibly build.

From price data (daily): adjusted close, volume, high, low, open.

From fundamentals (quarterly/annual): total assets, total equity (book value), total liabilities, net income, revenue, operating income, cash flow from operations, shares outstanding, dividends paid.

From market metadata: GICS sector, GICS industry, market capitalization (= shares outstanding $\times$ price).

From macro (for Person C's regime module): VIX ($\sim$ VIX), DGS10, DGS2, DBAA, DAAA, DFF (all FRED).

Q11. What about exotic data sources?

Do not use them. Analyst estimates, options-implied volatility, news sentiment, social media signals — all interesting, all out of scope for a 5-week project. They add complexity without strengthening the CV story.

5. File Layout and Interface

Q12. What files does the database consist of?

File	Contents	Shape (approx.)
<code>data/prices.parquet</code>	Daily adj. close, wide format	$\sim 5000 \text{ days} \times 1000 \text{ tickers}$
<code>data/fundamentals.parquet</code>	Quarterly fundamentals, long format	(date, ticker) MultiIndex
<code>data/constituents.parquet</code>	Month-end index membership	(date, ticker, in_index)
<code>data/macro.parquet</code>	Daily macro series	$\sim 5000 \text{ days} \times 6 \text{ columns}$

Q13. What interface does Person B consume?

A `data_loader.py` module exposing functions like `load_prices()`, `load_fundamentals()`, `load_macro()`, and `load_constituents()`. These return clean pandas objects with the reporting lag already applied. Agreed on Day 1 and not changed thereafter.

6. Universe Filter at Each Rebalance

Q14. What's the final rule for "is ticker X in this month's universe?"

A ticker is investable on rebalance date t if *all four* of the following hold:

1. It was an S&P 500 constituent on t (from `constituents.parquet`).
2. It has a non-NaN price on t (still actively trading).
3. It has non-NaN values for every factor the model uses (sufficient history).
4. The model produced a non-NaN prediction for it.

If any condition fails, the stock is skipped that month. No imputation. No substitution. This is not a bug — this is how a real investor behaves.

7. Mandatory Sanity Checks Before Models Are Trained

Q15. How do I know the pipeline is correct?

Before Person B trains a single model, Person A must run three tests on the full pipeline. All three must pass; otherwise *every* number the project produces is wrong.

1. **Random-prediction test.** Feed random predictions into the backtest. Sharpe must be approximately zero. If it's materially above zero, there's a bug — likely look-ahead.
2. **Oracle test.** Feed *actual future returns* in as "predictions." Sharpe must be very large (5+). Confirms the pipeline signs trades correctly.
3. **Uniform-prediction test.** Predict the same value for every stock on every date. Strategy return must be (approximately) zero. Confirms no unexpected dependencies on prediction magnitudes.

Q16. What about the data itself?

- Plot the number of stocks in the universe over time. Expected: roughly constant around 500, with gentle drift. Sudden jumps mean data bugs.
- Plot the distribution of monthly returns across the universe. Expected: symmetric around 0–1%, tails not absurd. Giant tails mean stock splits weren't adjusted for.
- Spot-check 5 famous events: Lehman (Sept 2008), COVID crash (Mar 2020), Tesla inclusion (Dec 2020), GME squeeze (Jan 2021), a known acquisition (pick one). Verify your data shows what you'd expect.

8. Summary Checklist (Person A, Week 1)

By end of Week 1, Person A can show:

Historical S&P 500 constituents file loaded into `constituents.parquet`.

Daily prices for ~1000 ever-in-S&P tickers pulled into `prices.parquet`, including delisted tickers, no forward-filling.

Quarterly fundamentals pulled into `fundamentals.parquet`, forward-filled within each stock, 45-day reporting lag enforced inside `data_loader.py`.

Macro data pulled into `macro.parquet`.

`data_loader.py` module with stable function signatures agreed with Person B on Day 1.

Universe-filter function that applies all four conditions from Q14.

Random / oracle / uniform sanity checks all pass.

Universe-size-over-time plot looks sane; famous-events spot checks pass.

*If the data pipeline is wrong, every Sharpe ratio in the report is a fiction.
Spend Week 1 getting it right; everything else compounds from there.*